EXPERIMENT NO. 06

Aim of the Experiment

To install MongoDB and perform basic CRUD (Create, Read, Update, Delete) operations using MongoDB tools.

Lab Outcome

By the end of this experiment, will be able to:

1. Install MongoDB successfully
2. Start and connect to MongoDB server
3. Understand collections and documents
4. Perform CRUD operations
5. Understand basic database structure

Introduction

MongoDB is a **NoSQL document-oriented database**.

Instead of: Tables,Rows,Columns

MongoDB uses: Databases,Collections,Documents

1 Collection A collection is a group of related documents.

Example: Students collection contains student records.

2 Document A document is a single record stored in JSON format.

Example: { "name":"John Doe", "age":22 "major":"Computer Science"}

It contains: Key (name) , Value (John Doe)

3 CRUD Operations

CRUD stands for:

Letter	Meaning
C	Create
R	Read
U	Update

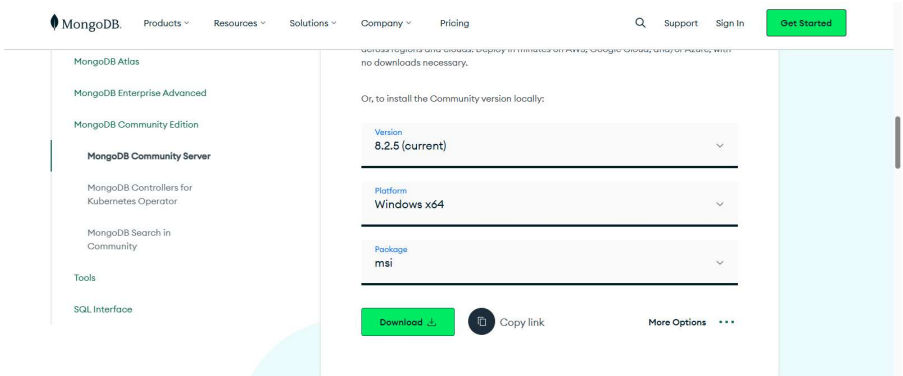
Letter	Meaning
D	Delete

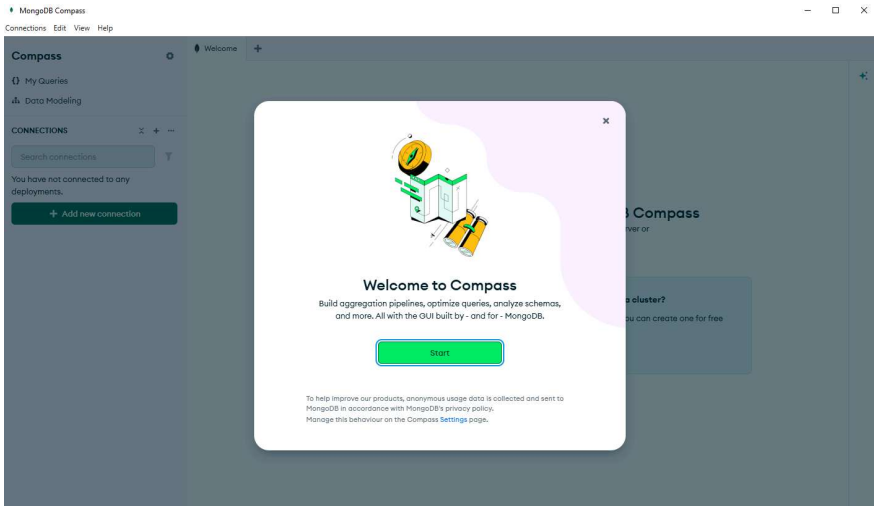
These are the 4 basic database operations.

IMPLEMENTATION STEPS

STEP 1: Install MongoDB

1. Go to official MongoDB website.
2. Download MongoDB Community Server.
3. Install it.
4. After installation, start MongoDB server.





To start server: mongod

If it runs successfully: MongoDB service is active.

STEP 2: Connect to MongoDB

Open another terminal and type: mongosh , You are now connected to MongoDB shell.

STEP 3: Perform CRUD Operations

1. CREATE OPERATION (Insert Data)

First select database: use test

If database does not exist: MongoDB creates it automatically.

```
test> use test
already on db test
```

Insert One Document

```
db.students.insertOne({
  name:"John Doe",
  age:22,
  major:"Computer Science"
});
```

Also Insert Multiple Documents

```
test> db.students.insertOne({
|   name: "John Doe",
|   age: 22,
|   major: "Computer Science"
| });
{
  acknowledged: true,
  insertedId: ObjectId('699fe688982cb727197c2907')
}
test> db.students.insertMany([
|   { name: "Jane Smith", age: 21, major: "Mathematics" },
|   { name: "Bob Johnson", age: 23, major: "Physics" }
| ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('699fe697982cb727197c2908'),
    '1': ObjectId('699fe697982cb727197c2909')
  }
}
```

2. READ OPERATION (Retrieve Data)

To see all documents:

db.students.find();

This displays all records.

With Condition

operators:

$gt → greater than

$lt → less than

$lte → less than or equal

```
test> db.students.find()
[
  {
    _id: ObjectId('699fe688982cb727197c2907'),
    name: 'John Doe',
    age: 22,
    major: 'Computer Science'
  },
  {
    _id: ObjectId('699fe697982cb727197c2908'),
    name: 'Jane Smith',
    age: 21,
    major: 'Mathematics'
  },
  {
    _id: ObjectId('699fe697982cb727197c2909'),
    name: 'Bob Johnson',
    age: 23,
    major: 'Physics'
  }
]
```

```
]
test> db.students.find().pretty()
[
  {
    _id: ObjectId('699fe688982cb727197c2907'),
    name: 'John Doe',
    age: 22,
    major: 'Computer Science'
  },
  {
    _id: ObjectId('699fe697982cb727197c2908'),
    name: 'Jane Smith',
    age: 21,
    major: 'Mathematics'
  },
  {
    _id: ObjectId('699fe697982cb727197c2909'),
    name: 'Bob Johnson',
    age: 23,
    major: 'Physics'
  }
]
```

```
]
test> db.students.find({ age: { $gte: 22 } })
[
  {
    _id: ObjectId('699fe688982cb727197c2907'),
    name: 'John Doe',
    age: 22,
    major: 'Computer Science'
  },
  {
    _id: ObjectId('699fe697982cb727197c2909'),
    name: 'Bob Johnson',
    age: 23,
    major: 'Physics'
  }
]
```

3. UPDATE OPERATION ;Used to modify existing data.

Update One Document : db.students.updateOne({ name:"John Doe" },

{ $set: { major:"Software Engineering" } }

);

Update Many Documents

```
test> db.students.updateOne(
|   { name: "John Doe" },
|   { $set: { major: "Software Engineering" } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.students.updateMany(
|   { age: { $gte: 23 } },
|   { $set: { status: "Senior" } }
| );
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

4. DELETE OPERATION

Used to remove documents.

Delete One

db.students.deleteOne({ name:"Bob Johnson" });

Deletes first matching document.

Delete Many

db.students.deleteMany({ age: { $gte:23 } });

Deletes all students age 23 or above.

```
test> db.students.deleteOne({ name: "Bob Johnson" });
{ acknowledged: true, deletedCount: 1 }
test> db.students.deleteMany({ age: { $gte: 23 } });
{ acknowledged: true, deletedCount: 0 }
test>
```

Conclusion:

MongoDB was successfully installed and the server (`mongod`) is running properly.
The data directory issue was fixed, allowing the database to start without errors.
To perform CRUD operations, MongoDB Shell (`mongosh`) must be installed separately.
Once `mongosh` is installed, you can connect to the server and execute database commands.